WoSM Academy

CS102-F26 · DIGITAL DESIGN AND COMPUTER ARCHITECTURE I

The Stepper, the Data Path and the Controller

Lecture 21: how the DX8 runs an instruction, step by step

Dr. Ali Khudiyev

Fall 2026

CC BY-SA 4.0

Today

01 The stepper

02 The data path

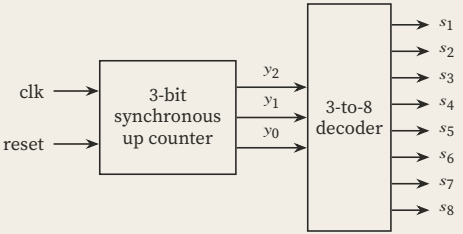
03 The controller

04 Summary

01

The stepper

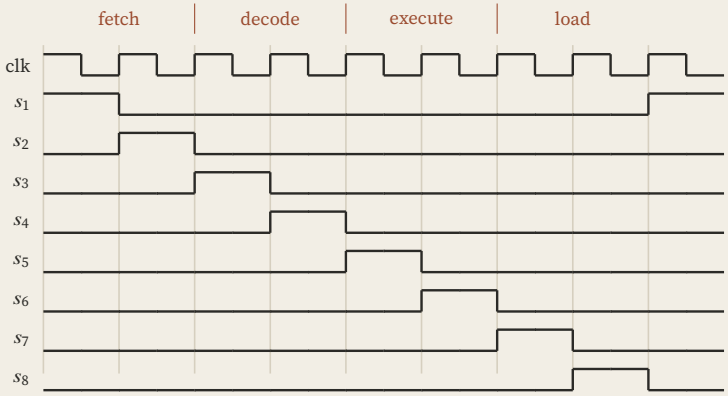
The stepper



After the book's Figure 6.5

- The stepper turns on one output at a time, in a fixed order, at each clock edge: its outputs are *one-hot* (Unit 8).
- Built from parts we know: a 3-bit synchronous up counter (Unit 13) and a 3-to-8 decoder (Unit 8).
- Its one input besides the clock is *reset*: the counter goes to 000, so the stepper goes to step 1.

Steps and stages



- Two steps per stage. After s_8 comes s_1 again, and the next instruction.

02

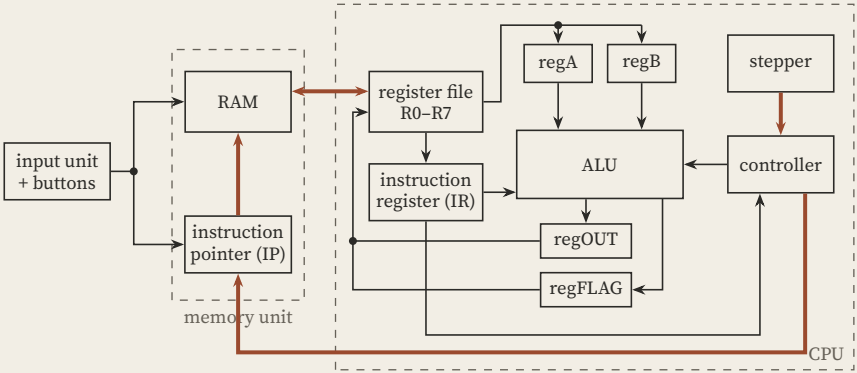
The data path

Which wires are hot?

- The *data path* is the set of wires along which data moves between the parts of the CPU and the memory.
- In each stage, only some of them carry data that matters. The book draws them in orange for each stage (Figures 6.8 to 6.11); here they are in red.
- To make data go from part A to part B, the controller read-enables A, write-enables B, and sets the select bits of the multiplexers on the way.

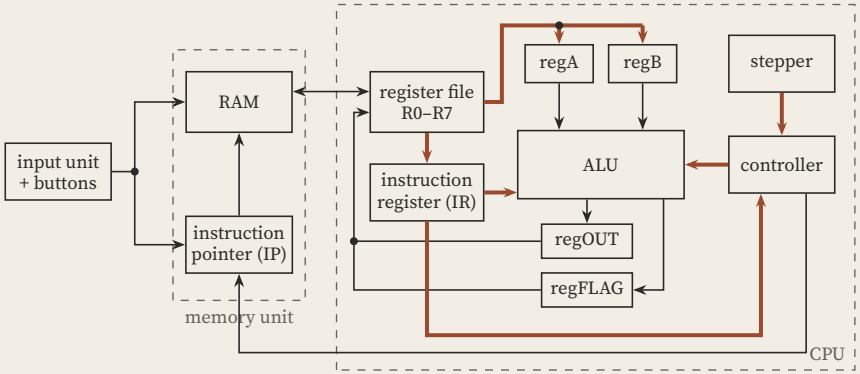
> **Remark**
>
> Every stage below also uses the stepper's output, so that the controller knows which step it is in.

Fetch



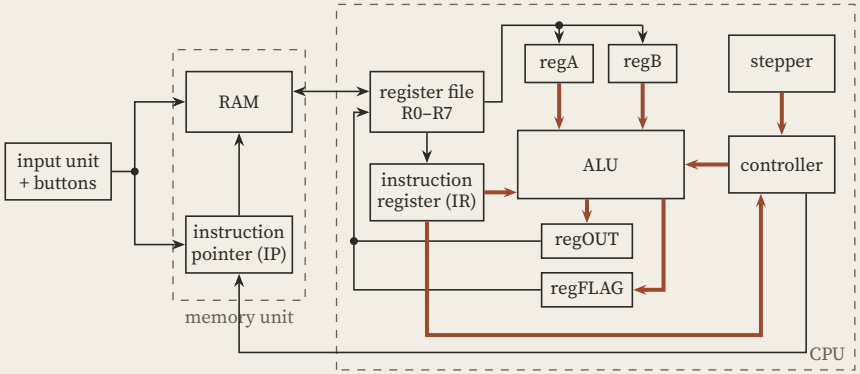
- The IP gives the RAM its address; the instruction comes from the RAM into the CPU. The controller also drives the IP.

Decode



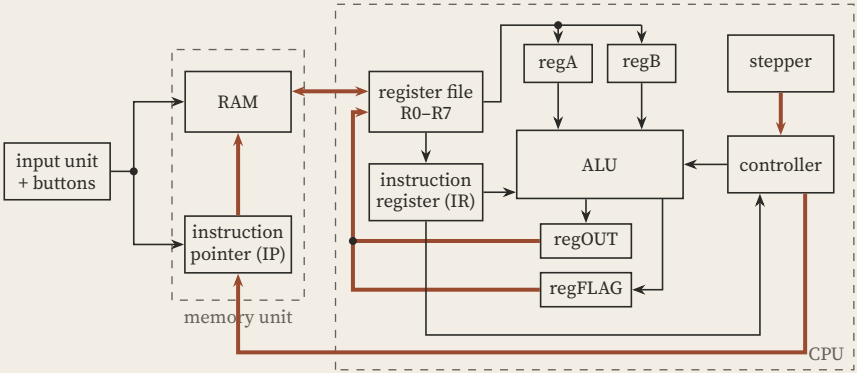
- The instruction is in the IR. Its arguments go to regA and regB, its operation, 4 bits, to the ALU; the controller reads it too.

Execute



- For an ALU instruction: regA and regB into the ALU, which computes what the IR asks for. The result goes to regOUT, the flags to regFLAG.

Load

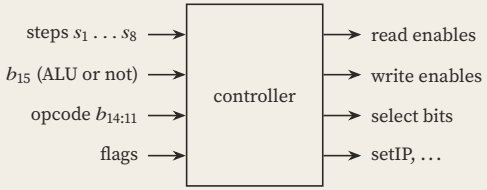


- regOUT goes to the register file; or a register goes to the RAM, at the address in the instruction. The controller moves the IP on to the next instruction.

03

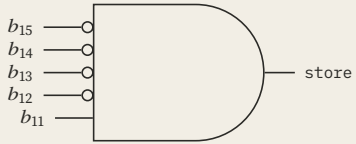
The controller

What the controller does



- Inputs: which step it is (from the stepper), the instruction (from the IR), and the ALU's flags.
- Outputs: the enable and select bits of every other part, for this step of this instruction.
- It is combinational: AND and OR gates, as in Lectures 6 to 8. The sequence in time comes from the stepper.

Recognising the instruction



1 exactly for 0 0001

- One AND gate per instruction: its inputs are b_{15} and the opcode, with a bubble on every bit that must be 0. It is a decoder (Unit 8), one output at a time.
- The book's controller recognises `move`, `arg1`, `arg2`, `store` and the four jumps this way.
- The ALU instructions need no such gate: their opcode goes straight to the ALU's multiplexer (Unit 18).

Some control signals

Signal	In the book's controller (Figure 6.12)
exec	$s_5 + s_6$: the execute stage
load	$s_7 + s_8$: the load stage
RAM_WE	`store` · load: the RAM is written only in a `store`'s load stage
load_aluOUT	b_{15}· load: an ALU result goes back in the load stage
setIP	`jmp` + `jmpgt`·gt + `jmplt`·lt + `jmpeq`·eq
regFileWE	1 in step s_1 (and others): the instruction comes from the RAM

- setIP is 1 when the IP must take the jump address `val`: always for `jmp`, and for a conditional jump only if its flag is set `(Unit 15)`.

Try it

1. In step s_6, which stage is the DX8 in? Which step follows s_8?
2. Draw the AND gate that recognises `arg1` (0 0011).
3. When is RAM_WE = 1?
4. The IR holds a `jmpeq` and eq = 0. What is setIP?
5. Which paths are hot while an `add` is executed?

Try it

1. In step s_6, which stage is the DX8 in? Which step follows s_8? Execute. Then s_1: the counter wraps around `(Unit 13)`.
2. Draw the AND gate that recognises `arg1` (0 0011).
3. When is RAM_WE = 1?
4. The IR holds a `jmpeq` and eq = 0. What is setIP?
5. Which paths are hot while an `add` is executed?

Try it

1. In step s_6, which stage is the DX8 in? Which step follows s_8?
   Execute. Then s_1: the counter wraps around `(Unit 13)`.
2. Draw the AND gate that recognises `arg1` (0 0011).
   Inputs $\overline{b_{15}}$, $\overline{b_{14}}$, $\overline{b_{13}}$, b_{12}, b_{11}.
3. When is RAM_WE = 1?
4. The IR holds a `jmpeq` and eq = 0. What is setIP?
5. Which paths are hot while an `add` is executed?

Try it

1. In step s_6, which stage is the DX8 in? Which step follows s_8?
   Execute. Then s_1: the counter wraps around `(Unit 13)`.
2. Draw the AND gate that recognises `arg1` (0 0011).
   Inputs $\overline{b_{15}}$, $\overline{b_{14}}$, $\overline{b_{13}}$, b_{12}, b_{11}.
3. When is RAM_WE = 1?
   In steps s_7 and s_8 of a `store` instruction, and never otherwise.
4. The IR holds a `jmpeq` and eq = 0. What is setIP?

5. Which paths are hot while an `add` is executed?

Try it

1. In step s_6, which stage is the DX8 in? Which step follows s_8?
   Execute. Then s_1: the counter wraps around `(Unit 13)`.
2. Draw the AND gate that recognises `arg1` (0 0011).
   Inputs $\overline{b_{15}}$, $\overline{b_{14}}$, $\overline{b_{13}}$, b_{12}, b_{11}.
3. When is RAM_WE = 1?
   In steps s_7 and s_8 of a `store` instruction, and never otherwise.
4. The IR holds a `jmpeq` and eq = 0. What is setIP?
   0: no jump, the program goes on to the next instruction.
5. Which paths are hot while an `add` is executed?

Try it

1. In step s_6, which stage is the DX8 in? Which step follows s_8?
   Execute. Then s_1: the counter wraps around `(Unit 13)`.
2. Draw the AND gate that recognises `arg1` (0 0011).
   Inputs $\overline{b_{15}}$, $\overline{b_{14}}$, $\overline{b_{13}}$, b_{12}, b_{11}.
3. When is RAM_WE = 1?
   In steps s_7 and s_8 of a `store` instruction, and never otherwise.
4. The IR holds a `jmpeq` and eq = 0. What is setIP?
   0: no jump, the program goes on to the next instruction.
5. Which paths are hot while an `add` is executed?
   regA, regB, the IR and the controller into the ALU; the ALU into regOUT and regFLAG; the IR into the controller.

04

Summary

Summary

- The stepper, a 3-bit counter and a 3-to-8 decoder, gives one-hot steps s_1 to s_8: two for each of fetch, decode, execute and load.
- The data path: the wires along which data moves. Each stage uses a different set of them.
- The controller is combinational. From the step, the instruction and the flags, it sets the read enables, write enables and select bits.
- Each instruction is recognised by one AND gate on b_{15} and the opcode.

Before the next lecture

- Read Sections 6.6.2 and 6.6.3 of the book.
- In Logisim-evolution, build the stepper from your counter and a decoder, and watch s_1 to s_8 in a timing diagram.
- Build the instruction-recognising AND gates and the setIP logic of the controller.

> **Next lecture** (Unit 22)
>
> Putting it all together: building and programming the DX8 computer.